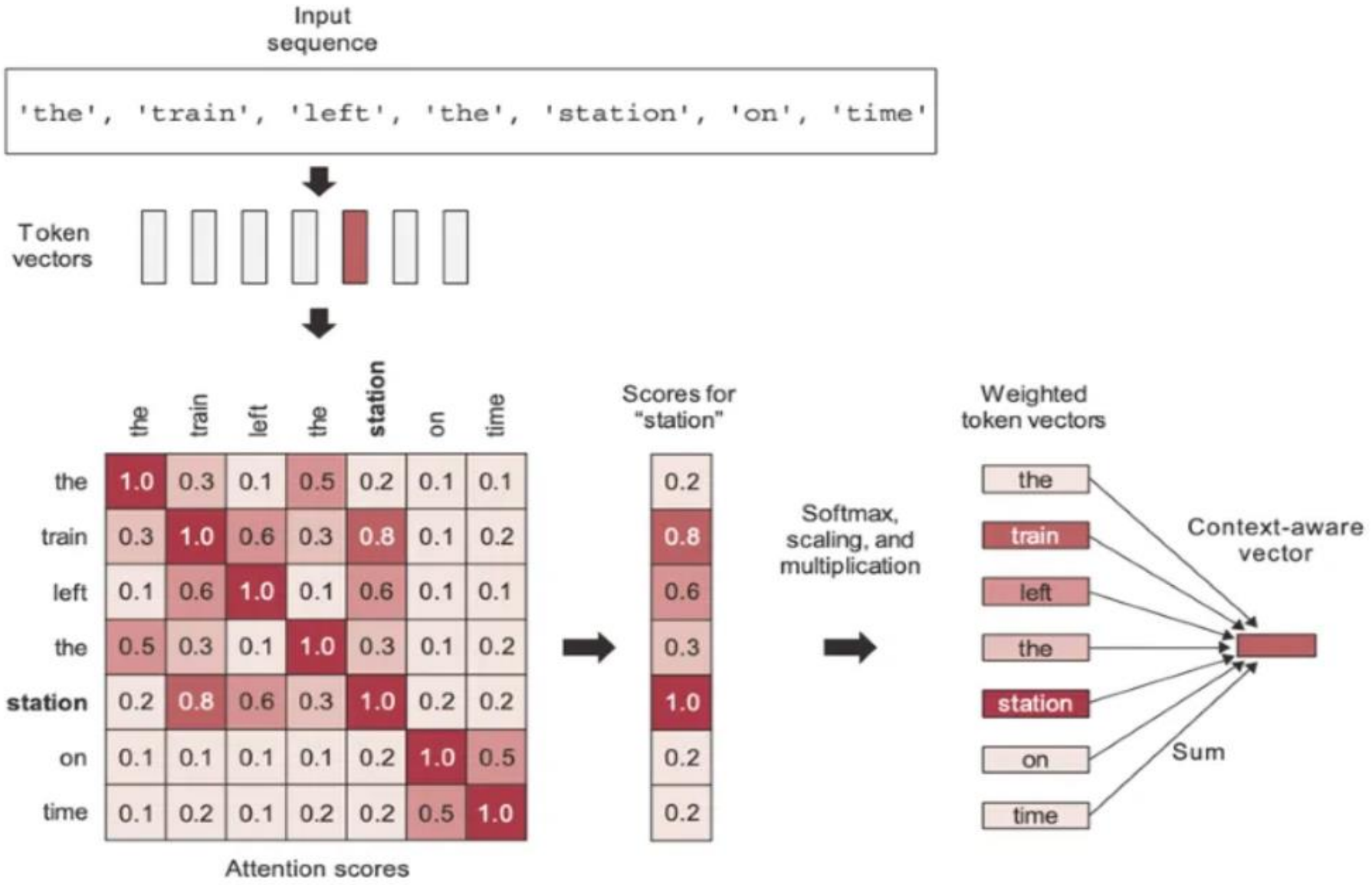
Transformer: Encoder

Multi-head attention and Encoder

Natural Language Processing

Ayesha Enayet

- ht
  d8


Input sequence
'the', 'train', 'left', 'the', 'station', 'on', 'time'
Token vectors
the train left the station on time
the 1.0 0.3 0.1 0.5 0.2 0.1 0.1
train 0.3 1.0 0.6 0.3 0.8 0.1 0.2
left 0.1 0.6 1.0 0.1 0.6 0.1 0.1
the 0.5 0.3 0.1 1.0 0.3 0.1 0.2
station 0.2 0.8 0.6 0.3 1.0 0.2 0.2
on 0.1 0.1 0.1 0.1 0.2 1.0 0.5
time 0.1 0.2 0.1 0.2 0.2 0.5 1.0
Attention scores
Scores for "station"
0.2
0.8
0.6
0.3
1.0
0.2
0.2
Softmax, scaling, and multiplication
Weighted token vectors
the
train
left
the
station
on
time
Context-aware vector
Sum

Self Attention

- Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations.

Table 1: Comparison of Self attention and RNN based Attention

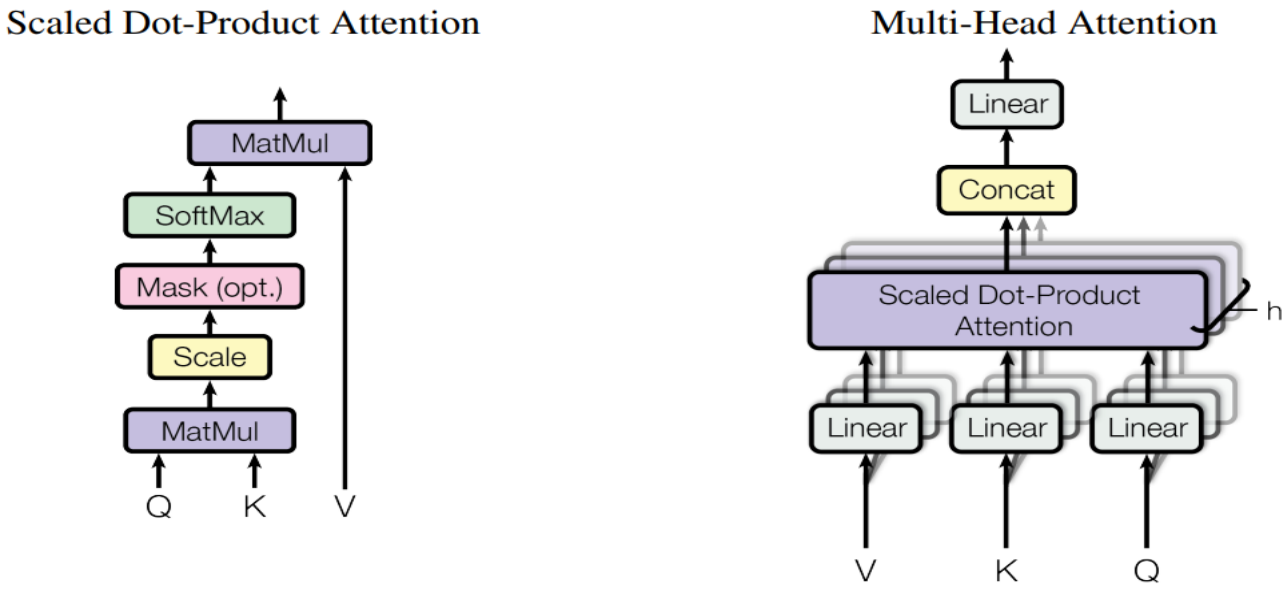
RNN Based Attention Mechanism	Self-Attention Mechanism
score: $e_{i,j} = f(s_{i-1}, h_j)$	**score** $= x_i^T x_j$ [Dot product of input vector with itself.]
Attention Score (SoftMax of score**):** $\alpha_{i,j} = \frac{exp(e_{i,j})}{\sum_{k=1}^{T} exp(e_{i,k})}$	**Attention score:** $\alpha_{i,j}$ = SoftMax(score/const.) = SoftMax $(\frac{x_i^T x_j}{const})$ [Dividing by a constant to control the variance.]
Context Vector: C(i)= $\sum_{j=1}^{T_x} \alpha\ (i,\ j)\ h(j)$	**Self-Attention Vector**: A(i)= $\sum_{j=1}^{T_x} \alpha\ (i,\ j) x_j$ = SoftMax $(\frac{x_i^T x_j}{const})$. x_j [in vectorized form]

Drawback

- The drawback of previously proposed self-attention:
  - We computed the Self-Attention vector, as derived above, based on the inputs of the vectors themselves.
  - In other words, there are no learnable parameters.

Solution

- “Attention is all you need”: **three trainable weight matrices are introduced and multiplied with input Xi separately, and three new terms Queries(Q), Keys(K), and Values(V)** come into the picture.



Solution

$shape\ of\ X \rightarrow (T \times d_{model})$

d_{model} = Embedding vector for each word (512 as per the paper)

$Queries$: $\boldsymbol{Q} = \boldsymbol{X}W^{Q}$; *Where* $\boldsymbol{W^{Q}}$ *is weight matrix introduced to calculate Q from X.*

$Keys$: $\boldsymbol{K} = \boldsymbol{X}W^{K}$; *Where* $\boldsymbol{W^{K}}$ *is weight matrix introduced to calculate K from X.*

Values: $\boldsymbol{V} = \boldsymbol{X}W^{V}$; Where $\boldsymbol{W^{V}}$ is weight matrix introduced to calculate V from X.

$$\textbf{\textit{Shape tracking}} \rightarrow (\textbf{\textit{shape of X}}) \ \textbf{\textit{Matrix Mult.}} (\textbf{\textit{shape of W}}^{-}) \rightarrow \textbf{Output shape}$$

$$Q = XW^{Q} \rightarrow (T \times d_{model}) * (d_{model} \times d_k) \rightarrow (T \times d_k)$$

$$K = XW^{K} \rightarrow (T \times d_{model}) * (d_{model} \times d_k) \rightarrow (T \times d_k)$$

$$V = XW^{V} \rightarrow (T \times d_{model}) * (d_{model} \times d_v) \rightarrow (T \times d_v)$$

Finally, substituting Q, K, V, and const. = $\sqrt{d_k}$ (d_k dimensionality of the key vector) in above self-attention eqn. (I), we get equation for attention as given below:

$$\textbf{Attention (Q, K, V) = SoftMax } \left(\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d_k}}}\right)\textbf{V} \quad \text{--------- (II)}$$

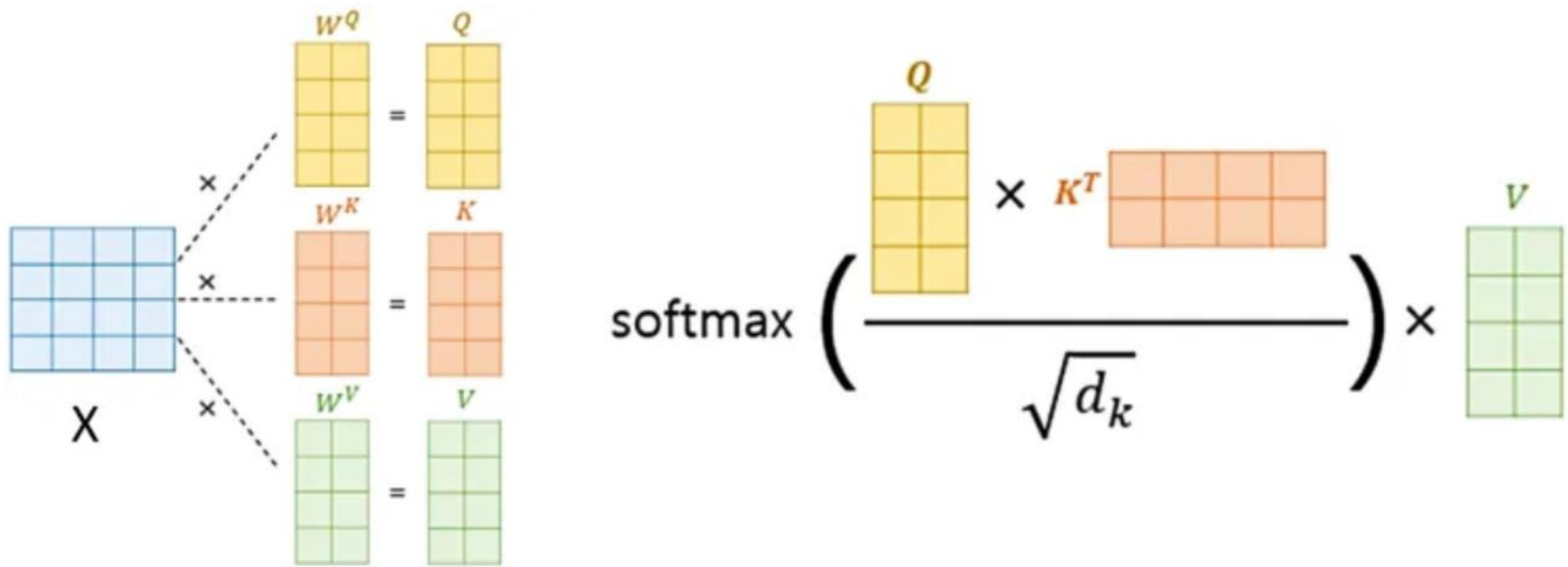


Figure 5. (Ref.)

Shapes as per the paper

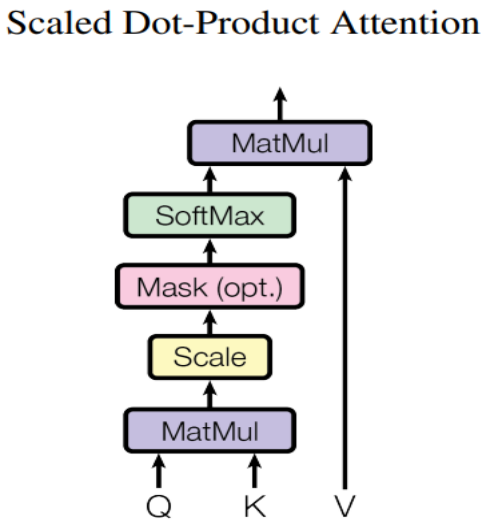
Object	*Shape*	*values*
q_i, k_i	d_k	$(64,)$
v_i	d_v	$(64,)$
x_i	d_{model}	$(512,)$
W^Q, W^K	$d_{model} \times d_k$	$(512, 64)$
W^V	$d_{model} \times d_v$	$(512, 64)$

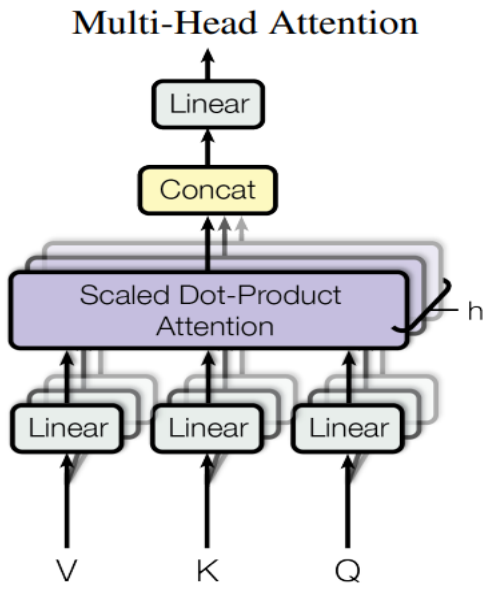
Shape of $QK^T \rightarrow (T \times d_k) \times (d_k \times T) \rightarrow (T \times T)$

And finally, the shape of final attention output: $(T \times T) \times (\mathrm{T} \times d_v) \rightarrow (\mathrm{T} \times d_v)$

Multi-Head Attention

- “Attention is all you need”: **three trainable weight matrices are introduced and multiplied with input Xi separately, and three new terms Queries(Q), Keys(K), and Values(V)** come into the picture.





Shape of $QK^T \rightarrow (T \times d_k) \times (d_k \times T) \rightarrow (T \times T)$

And finally, the shape of final attention output: $(T \times T) \times (\mathrm{T} \times d_v) \rightarrow (\mathrm{T} \times d_v)$

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

T X hd_v

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, ..., \mathrm{head}_\mathrm{h})W^O$$

$$\text{where } \mathrm{head}_\mathrm{i} = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

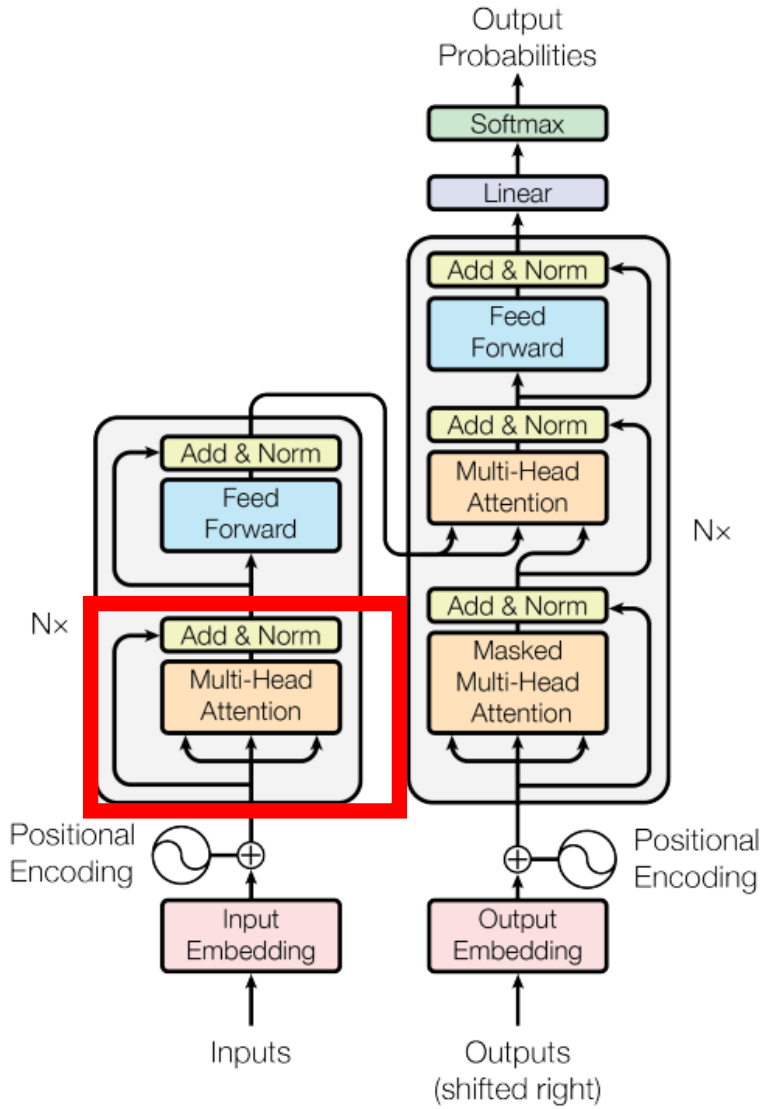


Figure 1: The Transformer - model architecture.

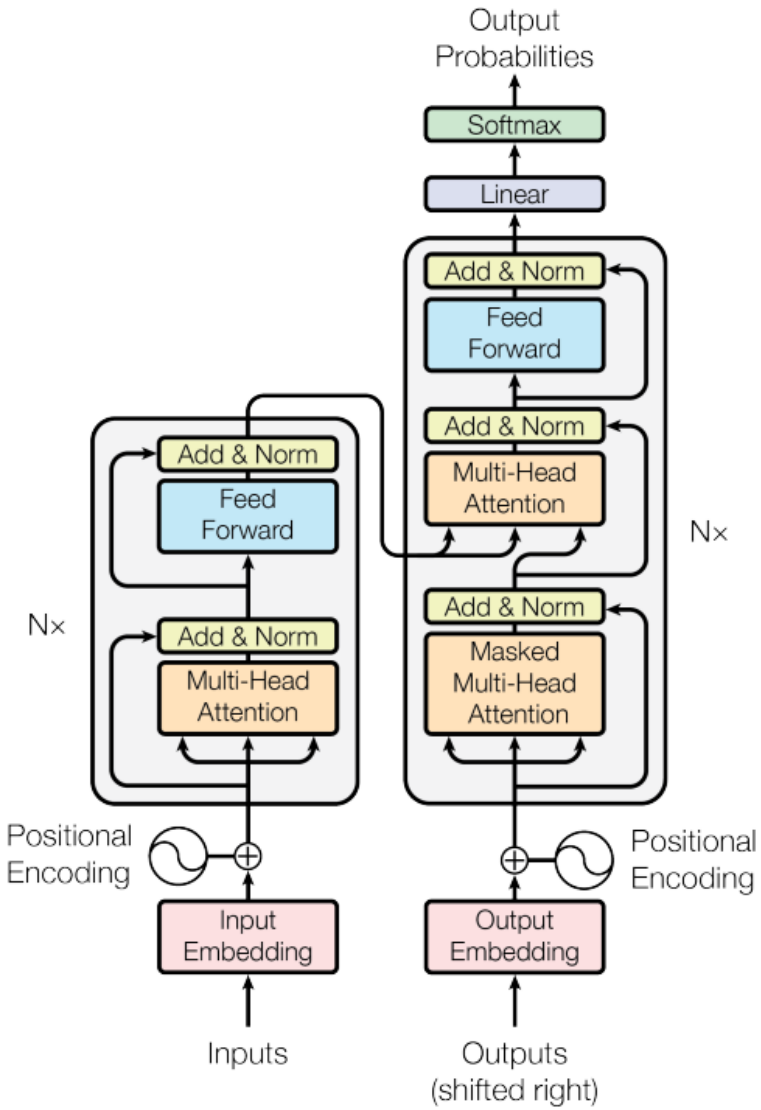


Figure 1: The Transformer - model architecture.

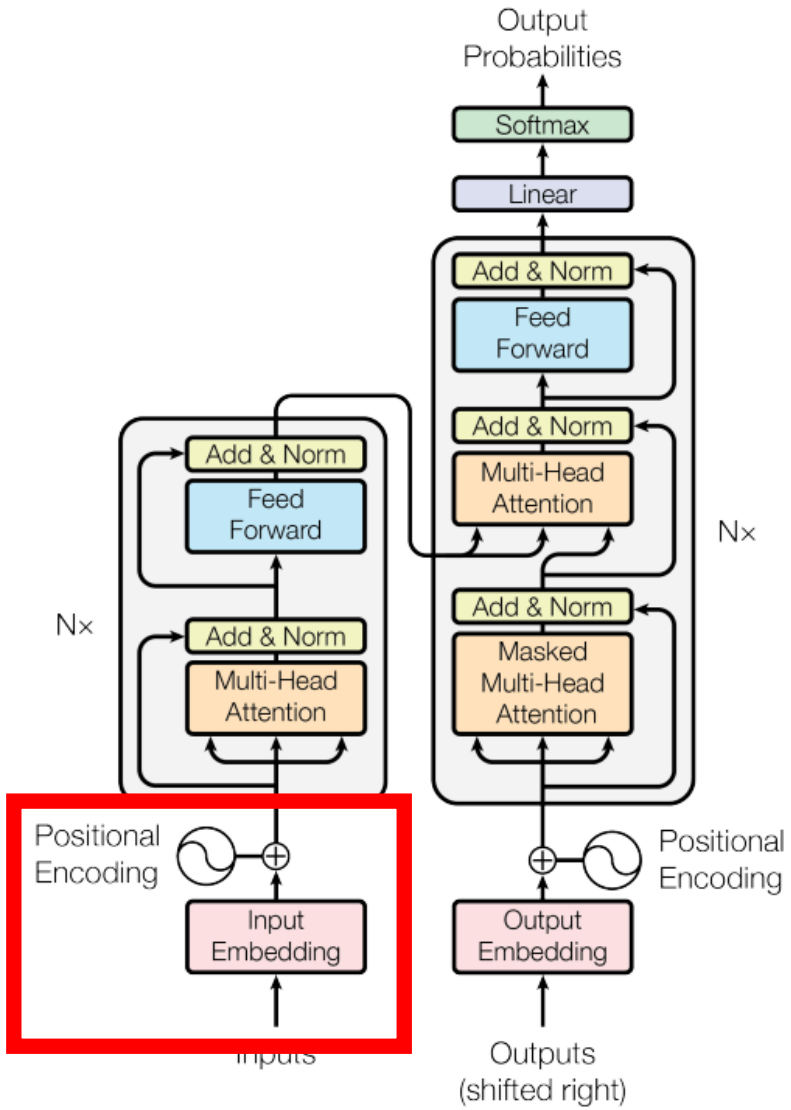


Figure 1: The Transformer - model architecture.

Positional Embedding

“Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed.” [From the original paper]

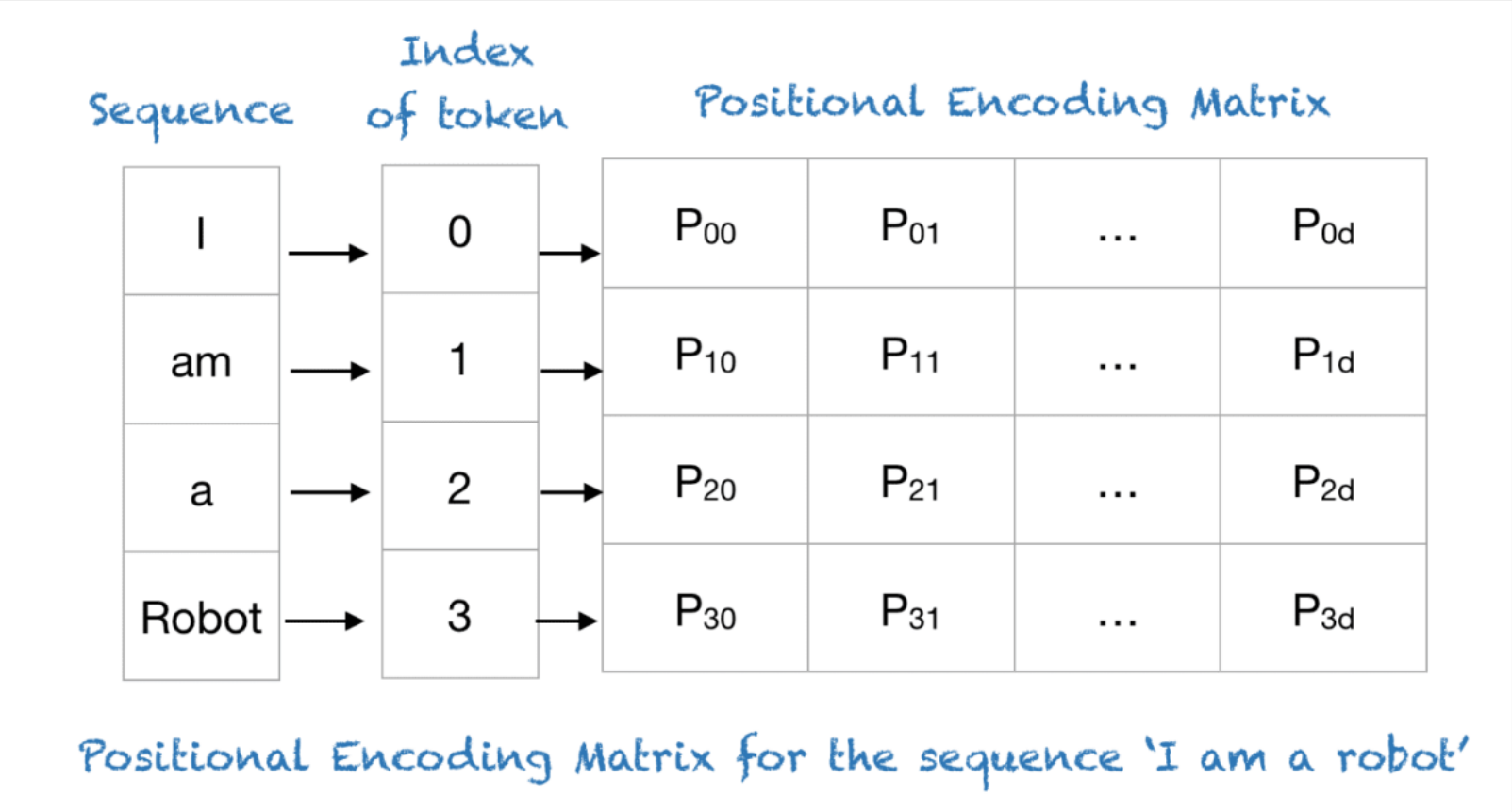
Positional Embedding

In these formulas:

- **pos** is the position (index) of the token in the sequence.
- **d** is the total dimension of the embedding vector.
- **i** represents the specific index within the embedding vector for which the sine or cosine function is being applied.

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$



https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/

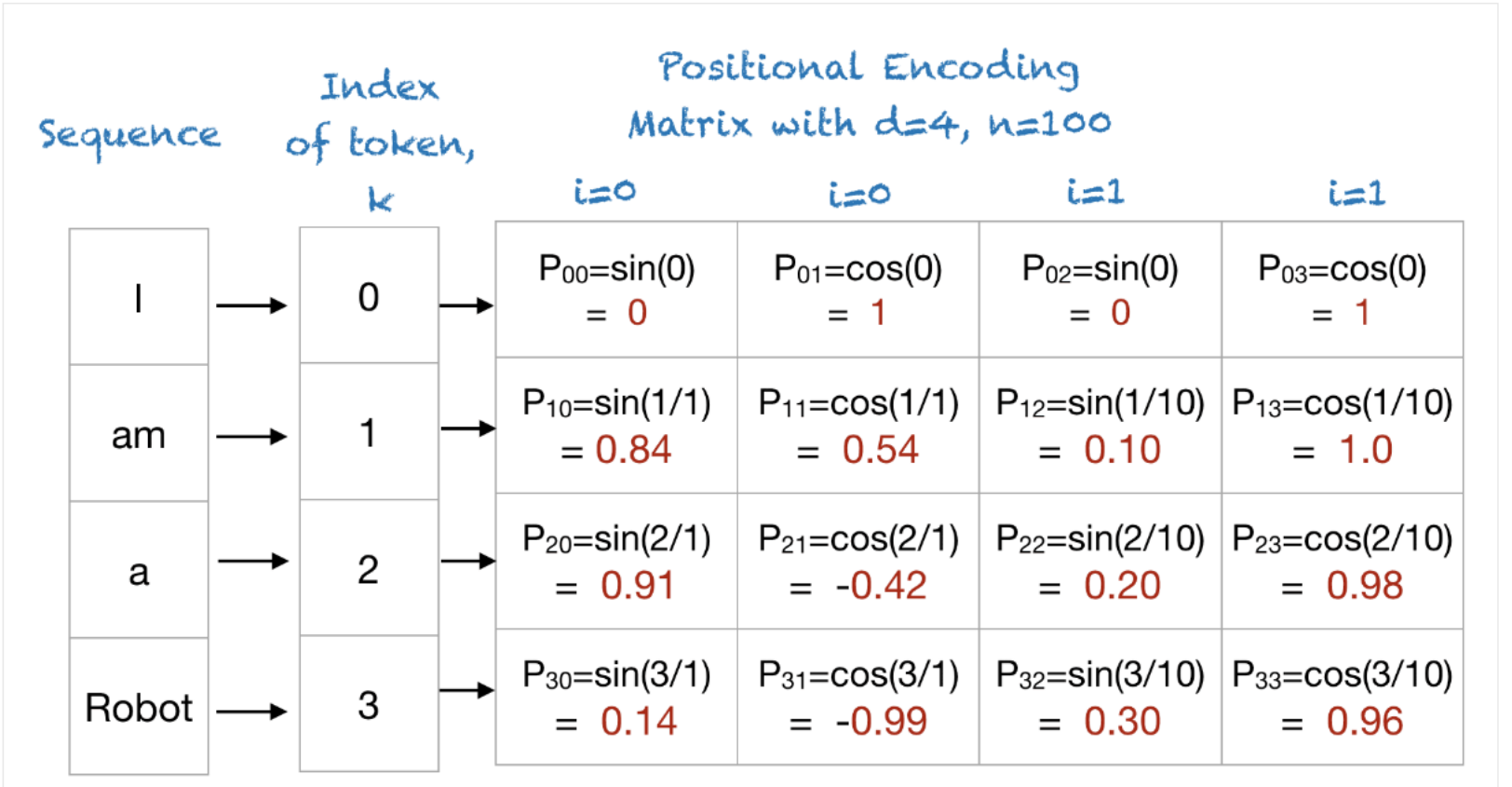
The positional encoding is calculated for each index **i** (which ranges from 0 to **d-1**) within the embedding vector for a given position **pos** in the sequence.

- For **even** dimensions (when **i** is even), the formula uses a sine function:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

- For **odd** dimensions (when **i** is odd), the formula uses a cosine function:

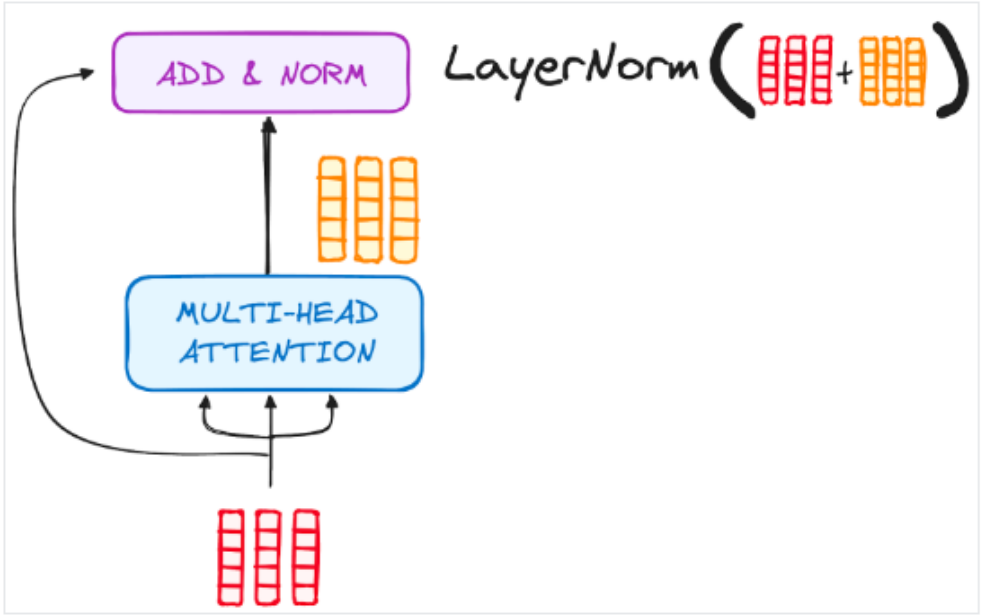
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$


Sequence
Index of token, k
Positional Encoding Matrix with d=4, n=100
i=0
i=0
i=1
i=1
I
0
P00=sin(0) = 0
P01=cos(0) = 1
P02=sin(0) = 0
P03=cos(0) = 1
am
1
P10=sin(1/1) = 0.84
P11=cos(1/1) = 0.54
P12=sin(1/10) = 0.10
P13=cos(1/10) = 1.0
a
2
P20=sin(2/1) = 0.91
P21=cos(2/1) = -0.42
P22=sin(2/10) = 0.20
P23=cos(2/10) = 0.98
Robot
3
P30=sin(3/1) = 0.14
P31=cos(3/1) = -0.99
P32=sin(3/10) = 0.30
P33=cos(3/10) = 0.96

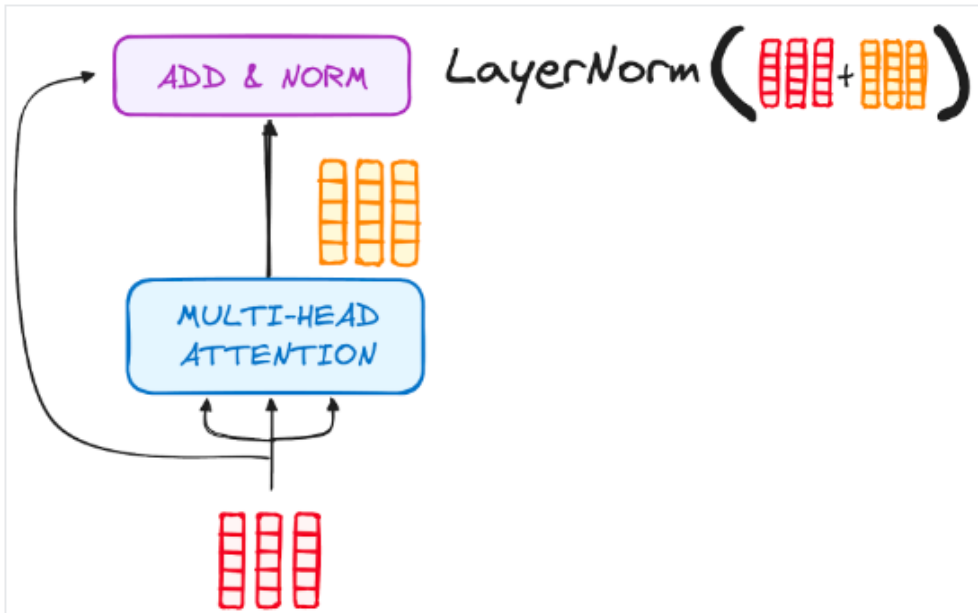
- What should be the dimension of positional embedding?

Add & Norm

Output is added to its input (residual connection) to help mitigate the vanishing gradient problem, allowing deeper models.



https://www.datacamp.com/tutorial/how-transformers-work



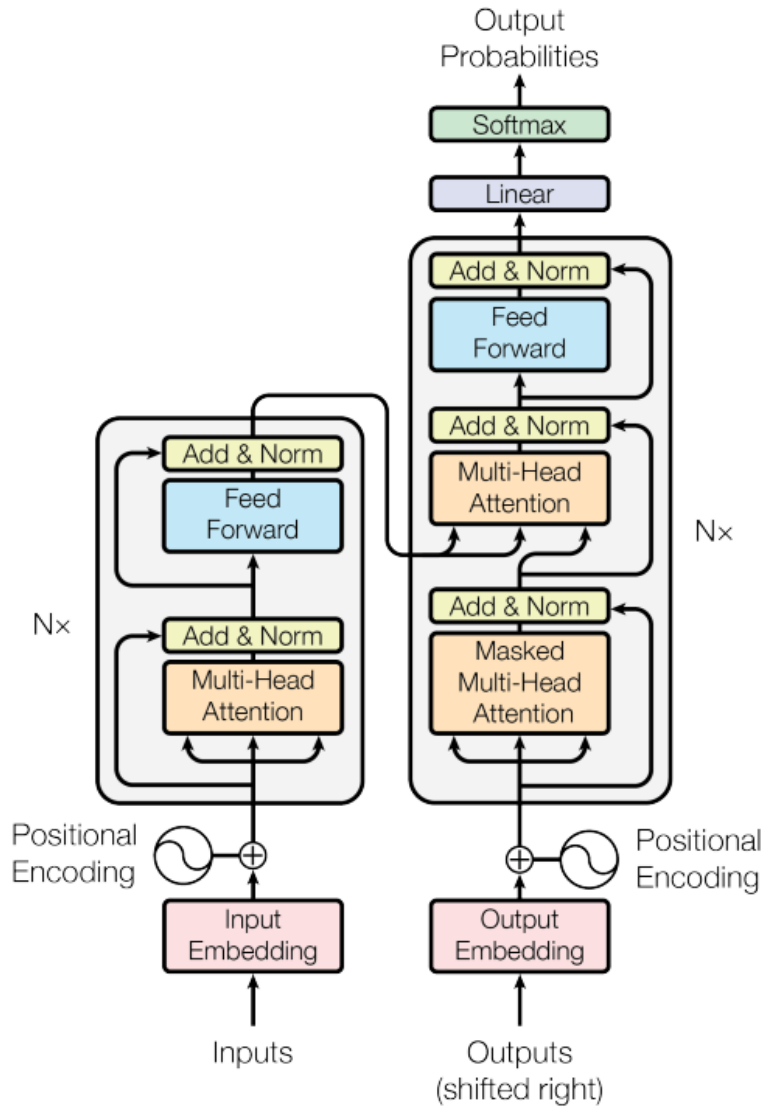
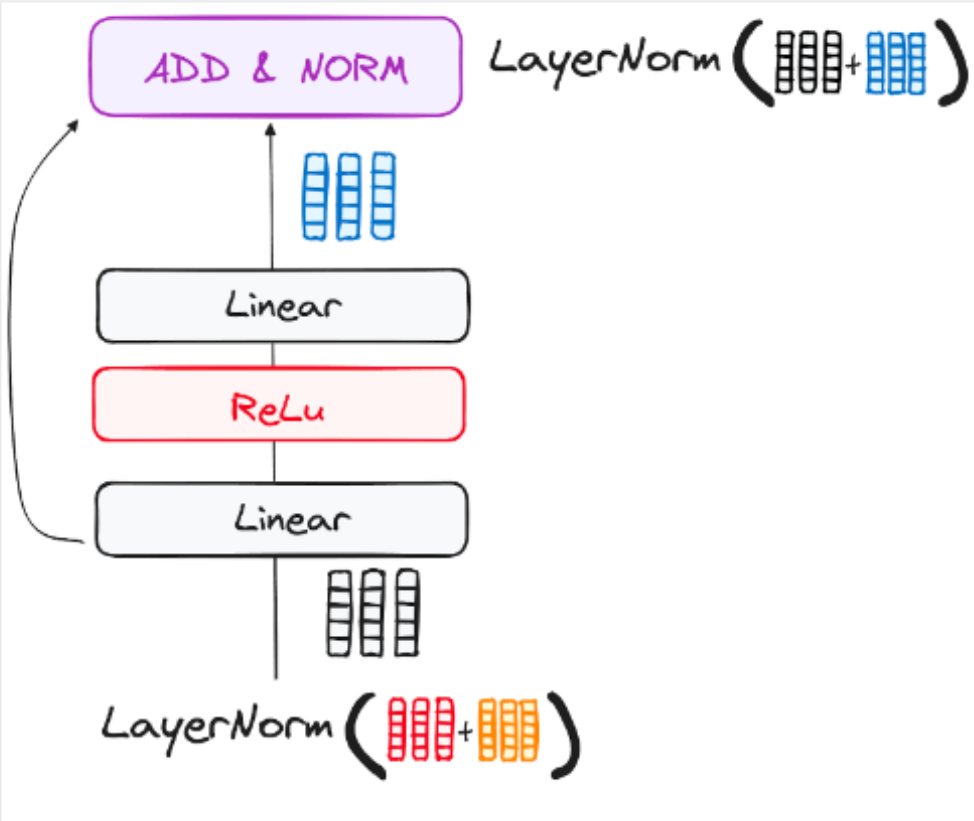


Figure 1: The Transformer - model architecture.

Feed-Forward



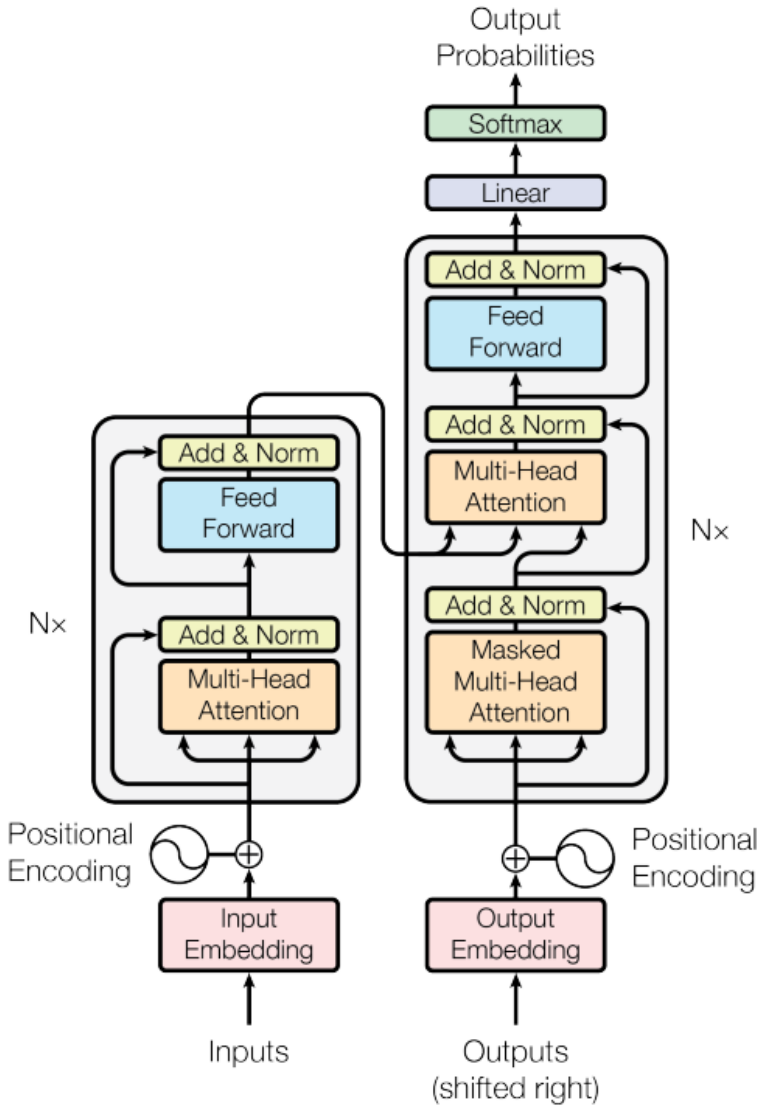
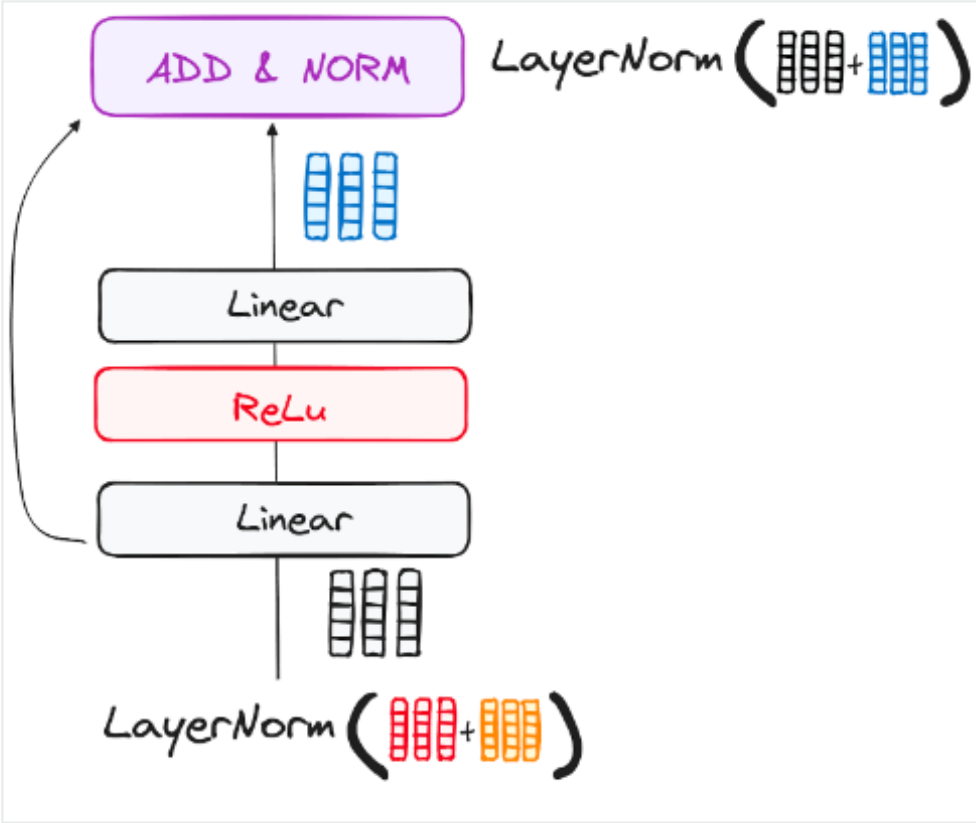


Figure 1: The Transformer - model architecture.

Feed-Forward

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$



Credit

- https://www.youtube.com/watch?v=QCJQG4DuHT0
- https://medium.com/@ramendrakumar/self-attention-d8196b9e9143
- https://medium.com/@geetkal67/attention-networks-a-simple-way-to-understand-self-attention-f5fb363c736d